

Non Repudiation and Token Verification

IFT 458 student explanation and JWT website exercise

How does a server know it issued a token? The answer depends on the token design. This project recognizes a random token by a trusted database record. A server using signed JWTs checks a cryptographic signature with a trusted key. Neither result, by itself, proves that a particular person performed a later action.

What non repudiation means

Non-repudiation concerns evidence that can support a claim about who originated data or performed an action if that claim is later disputed. Digital signatures can support such evidence when the signing key is reliably tied to an identity and appropriately controlled. A successful login alone is not that evidence. NIST describes non-repudiation in terms of verifiable origin and integrity, including verification by a third party. [1]

Keep these questions separate

Concept	Question it answers
Authentication	Which account does the presented credential identify?
Authorization	May that account perform this operation?
Integrity	Has the protected information changed?
Confidentiality	Who can read the information?
Non-repudiation	What evidence supports attribution if someone disputes the action?

A simple example

Alice logs in and receives a bearer token. Someone sends DELETE /games/123 with that token. The server can associate the credential with Alice's session. It cannot conclude from the token alone that Alice personally sent the request: the token could have been copied, stolen, or used by software acting for her.

What you will do

Trace this project's token lookup, compare it with JWT signatures, and use a public demonstration JWT at JWT.io to observe verification success and failure. Then explain what those results prove and what additional evidence a disputed action would need.

Use demonstration tokens only. Do not paste a real application token, password, shared signing secret, or private signing key into a public website. The supplied application token is not a JWT and cannot be verified by JWT.io.

1 How this server recognizes its own token

The current Game Score Tracker uses opaque bearer sessions. There is no JWT signature and no JWT signing key in this authentication flow. The following excerpts come from the local project; comments explain the omitted context.

When the user logs in

```
// controllers/authController.js after password verification
const token = randomBytes(32).toString('hex');
const expiresAt = new Date(Date.now() + 24 * 60 * 60 * 1000);
await Session.create({
  tokenHash: createHash('sha256').update(token).digest('hex'),
  user: user._id,
  expiresAt
});
```

The server generates a random token, records its hash with the user ID and expiry, and returns the raw token in the login response. The stored record is the server's memory of issuing that credential. Password validation happens first.

When the client requests a resource

Authorization: Bearer <token returned by login>

```
// middleware/authenticate.js after parsing the header
const tokenHash = createHash('sha256').update(match[1]).digest('hex');
const session = await Session.findOne({
  tokenHash, expiresAt: { $gt: new Date() }
});
if (!session) return deny();
const user = await User.findById(session.user);
if (!user) return deny();
```

The lookup above is reformatted across lines for readability. The middleware accepts only a matching, unexpired session for an existing user. It then sets req.user and calls next(). A changed or invented token normally has no matching record and receives 401.

Where the trust comes from

Trust comes from the protected session database and the controlled issuance code. SHA-256 is not a digital signature and does not identify a creator by itself. Anyone can hash a value; an attacker would also need a matching trusted session record or an already valid token. Database write compromise would undermine this design.

The check recognizes a credential issued by the trusted application, potentially by another instance sharing the database. It does not identify which physical server issued it or which human is now presenting it. A stolen valid bearer token can still pass until it becomes invalid.

Check your understanding

Why can the server reject a made-up 64-character token even though that token has the correct format? Why would a stolen valid token behave differently? Use tokenHash, expiresAt, and user in your answer.

2 How JWT signatures establish issuer trust

For comparison, a signed JWT normally contains three dot-separated parts: an encoded header, encoded claims, and a signature. Encoding makes the first two parts readable; it does not make them secret. The signature protects the signed bytes against undetected changes. [2]

`encodedHeader . encodedPayload . signature`

Two different ways to protect a JWT

Mechanism	Issuer does	Verifier does
HS256 shared key	Computes an HMAC with a shared secret.	Checks the HMAC with the same secret.
RS256 key pair	Signs with a private key kept by the issuer.	Verifies with the issuer's trusted public key.

With HS256, every party holding the shared secret can create a valid token. A match establishes membership in that trusted key-sharing group, not which member created it. With RS256, possession of the public key permits verification but not creation of new signatures. Issuer attribution depends on a trustworthy binding between the public key and the issuer, and on private-key protection.

A secure verification sequence

1. Use a maintained JWT library and configure the allowed algorithm in the server. Do not let an untrusted alg value freely choose the verification method.
2. Select a verification key from a trusted configuration or a trusted issuer's key set. A kid is a key identifier, not proof of identity; do not trust arbitrary key URLs supplied by a token.
3. Verify the signature over the original encoded header and payload. Reject a mismatch before using the claims.
4. Apply the token profile's claim checks, including expected issuer and audience and required expiration. Enforce nbf when present, account status, and the application's authorization rules. A valid signature is only one acceptance condition. [3]

What a verified signature does not establish

A server-signed JWT says that a trusted key holder signed these claims. It does not mean the user signed a later request, that the token is currently acceptable for this API, or that the request presenter is the original recipient. Bearer-token theft remains relevant.

A signature also does not encrypt the payload. HTTPS protects the token during transport, but software at the endpoints can read it. Authentication, issuer trust, and user-action non-repudiation are separate properties.

Applied to this project

The current middleware expects a 64-character hexadecimal session token. Sending a JWT to it fails the format check. This comparison is an explanation and website exercise, not a conversion of the project to JWT authentication.

3 Verify a demonstration token on JWT io

Use the website's built-in HS256 sample. This exercise checks a public example, not your application credentials. The current interface has separate JWT Decoder and JWT Encoder tabs; labels may change. [4]

1. Open <https://www.jwt.io/> and choose JWT Decoder. Use the displayed example or Generate example. Confirm the decoded header identifies HS256.
2. Read the decoded payload. Record which claims are present. Decoding is possible without establishing trust, so readable JSON alone is not evidence of a valid signature.
3. In JWT Signature Verification, use the public sample's matching Secret. The checked sample uses a-string-secret-at-least-256-bits-long as literal text, with Base64URL Encoded switched off. For a different generated example, use its matching demonstration secret.
4. Observe Signature Verified. Change the first character of the demonstration secret from a to b while leaving the encoded token unchanged. Observe Invalid Signature, then restore the original secret. Both outcomes were checked on the website for this guide.
5. Copy the original encoded token before testing tampering. In JWT Decoder, change the first character of the third dot-separated segment to a different Base64URL character. Keep the first two segments unchanged. The header and payload should remain readable, but signature verification must fail. Restore the original afterward.
6. Do not edit payload claims in JWT Encoder and call that a tampering test. The encoder may generate a new signature with the supplied key. A correctly re-signed token can verify because the signer has the key.

Record what each result means

Test	Expected observation	Your result
Original example and matching secret	Signature Verified	_____
Same token and wrong secret	Invalid Signature	_____
Changed signature segment	Verification failure	_____
Decoded JSON without a trusted key	Readable claims only	_____

Check claims separately

The built-in sample inspected for this guide has iat but no exp, iss, or aud. It therefore cannot demonstrate an application policy requiring expiry, issuer, and audience. Do not treat its green signature indicator as proof of those checks. With an instructor-provided demo, compare each claim to the intended API policy. A signed token with an expired exp must be rejected by that API even if its signature verifies.

A website verifies against the key you give it. It does not independently establish that the key belongs to your server or decide your application's permissions. Capture only the public example and your observations for coursework.

4 What stronger non repudiation requires

The following is a conceptual design for evidence about a disputed action; it is not implemented by this lab. A token check identifies a credential. Evidence about an action must also bind an actor to the exact content and context of that action.

1. Define the action precisely: operation, target record, material request fields, and intended service. Record a transaction identifier and a fresh challenge or nonce so the evidence is specific to one action.
2. Bind a signing key to the actor through an appropriate identity and key-registration process. Protect the private key. A server key identifies the server's signature, not the user's personal approval.
3. Have the actor's signing mechanism sign the defined action data and context. Define the byte representation consistently so verification covers exactly what was approved. Ordinary bearer-token attachment is not this step.
4. Verify the signature using the actor's trusted public key and evaluate freshness, key status, intended recipient, and authorization. Do not infer user intent solely from a background signature.
5. Preserve the signed data, verification result, key identity, relevant time evidence, and decision in protected audit storage. An ordinary checksum stored beside an editable log can be recomputed and is not independent proof of authenticity.
6. Make the evidence available for independent review when needed, including the identity binding and key-management history. Key compromise, shared devices, and delegated actions affect the strength of attribution. Cryptographic verification alone does not settle every dispute.

Questions to answer in your own words

- How does the current server recognize a token it issued?
- Why does changing a JWT claim without re-signing normally break verification?
- Why can both holders of an HS256 secret create valid tokens?
- Why is a server-signed JWT not proof that a user personally approved a deletion?
- What evidence would distinguish token issuance from approval of a particular action?

References and project evidence

[1] NIST CSRC. Non-repudiation glossary. https://csrc.nist.gov/glossary/term/non_repudiation

[2] JWT.io. Introduction to JSON Web Tokens. <https://www.jwt.io/introduction>

[3] RFC 8725. JSON Web Token Best Current Practices, sections 3.1, 3.8, 3.9, and 3.10. <https://www.rfc-editor.org/rfc/rfc8725.html>

[4] JWT.io. JSON Web Token Debugger. <https://www.jwt.io/> Interface and wrong-key exercise checked September 21, 2026.

Local implementation reviewed: `controllers/authController.js`, `middleware/authenticate.js`, and `models/sessionModel.js`. These files establish the opaque-session behavior described here.